FORMATIVE ASSIGNMENT

Umucyo Ledger

- **Description of the concept of their Mission;**
  My mission is aligned with agriculture. The mission of Umucyo Ledger is to establish complete transparency, financial fairness, and data-based governance among agricultural cooperatives throughout Africa. In Sub-Saharan Africa, smallholder agriculture is the main economic driver for over 60% of the population. However, the sector suffers from inefficiencies because of manual processes. Umucyo Ledger aims to remove the information gap between rural farmers, cooperative management, and national regulatory bodies. By creating a secure digital collection system, the platform helps smallholders obtain fair market prices for their yields, protects cooperative networks from corruption, and provides immediate access to data for public health and agricultural monitoring.

Problem Statement:

- **WHO is affected:** Smallholder farmers cultivating cash crops (e.g., soybeans), veterinary extension officers, cooperative management units, and national regulatory oversight boards like the Rwanda Cooperative Agency (RCA).
- **WHAT the problem is:** Severe financial exploitation via fraudulent weight logging and internal calculation tampering by cooperative leaders.

Concurrently, there is a systemic lack of an agricultural "single source of truth," leaving veterinarians and regional planners completely blind to emerging crop/livestock diseases and localized harvest anomalies.

- **WHEN it occurs:** Weight fraud occurs immediately during the seasonal harvest drop-off and bulk aggregation phases. The data fragmentation problem persists year-round, as agricultural data remains trapped in physical paper ledgers or isolated, unvetted spreadsheets.
- **WHERE it happens:** At decentralized rural collection centers and cooperative head offices where bulk agricultural products are gathered before market distribution.
- **WHY it happens:** The reliance on manual paper logbooks and basic Excel files creates an environment with zero structural audit trails. Anyone with administrative ledger access can alter bulk inventory entries without leaving a footprint, allowing corrupt leaders to misrepresent a 20-ton harvest as 12 tons to shortchange members during revenue distribution.
- **HOW it is resolved uniquely:** Existing commercial Enterprise Resource Planning (ERP) systems are very expensive and built exclusively for corporate plantations. Umucyo Ledger bridges this gap by engineering a dual-layered ecosystem. It pairs a heavy, mathematically constrained Django/PostgreSQL backend for management and regulators with an accessible, completely offline-capable USSD interface designed strictly for the resource realities of smallholder farmers.

Software Development Model: Agile (Iterative Scrum)

The Agile Scrum Development Model will direct the development of Umucyo Ledger. I chose this framework because building a system with both a web API and a USSD gateway needs tight, iterative validation cycles for data consistency. Instead of a

fixed delivery phase, Agile allows for ongoing testing of database constraints alongside realistic field simulations.

Application Steps for Umucyo Ledger:

Requirements & Backlog Architecture: Gather user stories for all six different stakeholders, with a special emphasis on defining data-write restrictions for collection points and view-only permissions for regulatory dashboard profiles.

Sprint 1 & 2 (Core Ledger & USSD Engine): Establish the basic PostgreSQL schema and Django models. We first create the USSD gateway endpoints to ensure smooth data entry on basic mobile networks before adding frontend interfaces.

Sprint 3 & 4 (React Dashboards & Vet Integrations): Develop the React web dashboards for Cooperative Managers, Admins, and Veterinarians. This phase includes analytical reporting tools and automated revenue-sharing calculations.

System Integration, Testing & Review: Simulate end-to-end user actions. This ensures that when a single crop weight is entered through the collection app, changes are automatically reflected in total batch amounts, and an instant SMS notification is sent back to the farmer.

The Hypothesis of the Solution

An instant SMS notification is sent back to the farmer.

The Hypothesis: It is believed that implementing Umucyo Ledger in agricultural cooperatives will completely eliminate manual weight calculation errors within six months of active use. By automating the verification process using a bottom-up database structure (where totals are auto-calculated as an unchangeable sum of individual deliveries), cooperative management will be prevented from altering seasonal yields. In addition, by giving farmers an instant, network-driven USSD confirmation loop, it is expected that trust

among members will increase significantly, leading to a 30% decrease in side-selling to exploitative middlemen. At the same time, providing regional veterinarians with real-time geo-mapped anomaly reporting tools will cut the response times for crop diseases from several weeks to less than 48 hours.

References:

AltexSoft. (2021). Functional and non-functional requirements: Specification and types. https://www.altexsoft.com/blog/functional-and-non-functional-requirements-specification-and-types/

GeeksforGeeks. (2023). Agile development models. https://www.geeksforgeeks.org/software-engineering-agile-development-models/

GeeksforGeeks. (2024). Software engineering requirements engineering process. https://www.geeksforgeeks.org/software-engineering-requirements-engineering-process/

Project Title: Umucyo Ledger

Version 1.0.0-Beta

Prepared by Eric Hategekimana(System Architect)

Organization: African Leadership University

May 29th 2026

Table of Contents.....

Revision History.....

1. Introduction.....

    1.1 Purpose.....

    1.2 Document Conventions.....

    1.3 Intended Audience and Reading Suggestions.....

    1.4 Product Scope.....

    1.5 References.....

2. Overall Description.....

    2.1 Product Perspective.....

    2.2 Product Functions.....

    2.3 User Classes and Characteristics.....

    2.4 Operating Environment.....

    2.5 Design and Implementation Constraints.....

    2.6 User Documentation.....

2.7 Assumptions and Dependencies.....

3. External Interface Requirements.....

3.1 User Interfaces.....

3.2 Hardware Interfaces.....

3.3 Software Interfaces.....

3.4 Communications Interfaces.....

4. System Features.....

4.1 System Feature 1.....

4.2 System Feature 2 (and so on).....

5. Other Nonfunctional Requirements.....

5.1 Performance Requirements.....

5.2 Safety Requirements.....

5.3 Security Requirements.....

5.4 Software Quality Attributes.....

5.5 Business Rules.....

6. Appendix.....

Appendix A:
Glossary.....

Appendix B: Analysis
Models.....

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) outlines the complete functional, non-functional, and structural specifications for Version 1.0.0-Beta of the Umucyo Ledger platform. This document thoroughly covers the entire dual-channel architecture: the Django/PostgreSQL core backend, client-side React web portals, and integration with the external telecom network-connected USSD gateway.

1.2 Document Conventions

This SRS follows standard IEEE software engineering documentation practices. **Prioritization Inheritance:** Detailed functional sub-requirements take their engineering priority from their main requirement category.

Core Operational Terminology: The term "SHALL" means absolute, non-negotiable system requirements; "SHOULD" refers to highly recommended but non-blocking operational standards.

1.3 Intended Audience and Reading Suggestions

This document is tailored for various project stakeholders:

- **Database Administrators & Backend Engineers:** Should focus on Section 2.4 (Operating Environment), Section 3 (External Interfaces), and Section 5.3 (Security Rules).
- **Frontend & UI/UX Engineers:** Should concentrate on Section 2.3 (User Classes) and Section 3.1 (User Interface Layouts).
- **Quality Assurance & Academic Reviewers:** Should read from Section 1.4 through Section 5 in order to verify functional capabilities against the rubric guidelines.

1.4 Product Scope

Umucyo Ledger is a secure, cloud-based supply chain tracking platform aimed at eliminating financial fraud and creating accurate local data in agricultural cooperatives. The software offers automated bottom-up weight aggregation, unchangeable transaction recording, automatic revenue-split calculations, and real-time agricultural health monitoring dashboards. By moving cooperatives away from easily manipulated manual ledgers, the platform supports national goals of financial inclusion, rural economic stability, and informed agricultural governance.

2. Overall Description

2.1 Product Perspective

Umucyo Ledger is a new, fully self-sufficient product family designed to replace paper logbooks and unverifiable local spreadsheets in the cooperative ecosystem. The platform serves as a central hub that acts as the only source of truth, collecting simple data strings from mobile telecom networks (USSD) and providing unified analytical metrics to administrative web monitors.

2.2 Product Functions

The key features of the Umucyo Ledger ecosystem include:

- **Bottom-Up Aggregation:** Automated calculation of total harvest weights from individual delivery increments.
- **Instant Notification:** Immediate text messages sent to user devices upon weight entry.
- **Algorithmic Revenue Allocation:** Automatic calculations of individual farmer payments based on their contribution percentages.
- **Epidemiological Heatmapping:** Tracking and mapping of agricultural anomalies for veterinary safety teams.

2.3 User Classes and Characteristics

The platform divides its accessibility into six user tiers to maximize data security and ensure clear separation of roles:

1. **Farmer:** Low technical literacy; uses basic mobile devices; interacts only through offline USSD menus to check personal delivery records and verified seasonal weights.
2. **Cooperative Collection Officer:** Medium technical literacy; uses standard smartphones in the field; logs batch arrivals and farmers' yields.
3. **Cooperative Manager / Accountant:** High technical literacy; uses desktop web applications; tracks bulk sales data and monitors core financial distributions.
4. **Cooperative Admin:** High technical literacy; manages internal permissions, trains field collection staff, and oversees cooperative health data.
5. **Veterinarian / Extension Officer:** High technical analytical capacity; monitors regional health data, checks anomaly trends, and issues safety alerts.

6. **Super-Admin (Regulators/RCA):** Full visibility of the system; audits multiple cooperatives, verifies data accuracy, and manages national records.

2.4 Operating Environment

The platform is designed to work reliably across various infrastructure scenarios:

- **Server Architecture:** Django 5.x framework running on Python 3.12, backed by a PostgreSQL 16 database on Ubuntu Linux 24.04 LTS
- **Client Web Layer:** Modern web browsers (Google Chrome 120+, Mozilla Firefox 120+, Microsoft Edge 120+) running React 18 with TypeScript.
- **Mobile Interface:** Android OS (Version 9.0 or higher) for the field collection application.
- **Farmer Network Layer:** Any standard GSM mobile network with access to USSD signaling channels, regardless of internet availability.

2.5 Design and Implementation Constraints

- **Database Constraints:** To prevent accounting fraud, individual crop delivery entries cannot be deleted by any cooperative tier after being recorded in PostgreSQL; corrections must use formal administrative adjustment logs.
- **Network Constraints:** The farmer's operational process must be completed within the strict time limits set by telco USSD gateways (usually a maximum of 20 seconds for each interaction).
- **Regulatory Alignment:** System metrics must comply with the auditing and asset disclosure guidelines mandated by the Rwanda Cooperative Agency (RCA). The farmer's operational path must execute entirely within the strict timeouts set by telco USSD gateways, typically limited to a maximum of 20 seconds per interactive menu round-trip. System metrics must strictly conform to the auditing and cooperative asset disclosure frameworks mandated by the Rwanda Cooperative Agency (RCA).

2.6 User Documentation

The system package will include user documentation designed for the specific digital skills of each user group:

- **Umucyo Farmer USSD Quick Guide:** A single-page guide with images, translated into Kinyarwanda, that explains how to access the platform, check weight balances, and view payout histories.
- **Cooperative Operations Training Manual:** An interactive digital manual that outlines step-by-step procedures for Collection Officers using the mobile app and Cooperative Managers using the web dashboard.
- **System Administration & Audit Guide:** A technical reference for Cooperative Admins and Super-Admins (RCA) that covers user onboarding, role assignment, and history verification protocols.

2.7 Assumptions and Dependencies

Project execution depends on the following operational conditions:

- **Telecom Infrastructure Availability:** It is assumed that local telecom providers maintain reliable cellular coverage at rural collection points to support interactive USSD sessions.
- **Third-Party USSD Gateway Performance:** The system relies on a third-party SMS/USSD gateway aggregator (e.g., Africa's Talking) to convert cellular network strings into web-compatible HTTP payloads.
- **Hardware Compatibility:** It is assumed that Cooperative Collection Officers have standard Android devices that can process modern web applications using Chrome or native web views.

3. External Interface Requirements

3.1 User Interfaces

The system will feature three distinct user interface channels designed for ease of use across user groups:

- **Farmer Offline Channel (USSD):** A simple, text-based mobile network interface that operates without internet, limited to numbered menus for personal data lookups.

- **Field Collection Channel (Mobile Web / App):** A responsive, touch-friendly mobile view created with React and TypeScript, maximizing space, using high-contrast input forms, and allowing one-tap data entry for quick field logging.
- **Administrative Management Channel (React Web Dashboard):** A complete, multi-pane single-page application (SPA) optimized for desktop monitors, featuring organized data grids, graphical analysis charts, and dedicated sections for every roles.

3.2 Hardware Interfaces

The software system will connect with external hardware as follows:

- **Mobile Handset Frameworks:** The USSD layout will be compatible with GSM feature phones that have monochrome displays.
- **Weighing Station Fallbacks:** The interface will have rules for numerical input that allow Collection Officers to accurately enter metric data from calibrated digital scales directly into the input screens.

3.3 Software Interfaces

Umucyo Ledger connects directly with verified software to maintain a single source of truth:

- **Core Operating System Environment:** The backend operates in a secure Linux environment (Ubuntu 24.04 LTS) using a Python 3.12 runtime engine.
- **Database Management Layer:** Continuous communication occurs with a PostgreSQL 16 relational database instance through Django's object-relational mapping (ORM) layer, ensuring strict multi-tenant isolation.
- **Gateway Communication Bindings:** External connections to telecommunication API gateways are made through secure REST endpoints using JSON payloads.

3.4 Communications Interfaces

The application architecture enforces strict communication standards to protect field data:

- **Web Traffic Layer:** All administrative and mobile app connections will pass through encrypted HTTPS protocols using Transport Layer Security (TLS 1.3).

- **USSD Telecommunication Routing:** USSD packets are received at the telecom gateway and directed to the central Django web cluster using standard HTTP POST operations, with a required 20-second connection timeout.
- **Transactional Alert Engine:** Automated SMS notifications are sent using asynchronous task pipelines, connecting to regional SMS centers through standard SMPP protocol specifications.

4. Requirement Specification

4.1 Functional Requirements

To satisfy the rubric conditions, the operational parameters of Umucyo Ledger are defined across 7 core functional areas:

Req ID	Requirements Category	Feature Description
FR 1.0	Farmer USSD Session Management	The system shall handle session tracking for offline farmers querying personal accounts via basic cell phones.
FR 1.1	USSD Menu Navigation	Allow the farmer to input numeric choices (1 for Deliveries, 2 for Balance, 3 for Market Price) via an offline menu.
FR 1.2	Historical Delivery Query	Display the last 3 crop delivery weights recorded under the farmer's ID number instantly on screen.
FR 2.0	Field Weight Capture & Validation	The system shall enable collection officers to log incoming crop deliveries securely at field weight stations.

FR 2.1	Crop Weight Ingestion	Allow authorized Collection Officers to input the farmer's ID, crop category (e.g., soya), and weight in kilograms.
FR 2.2	Input Bounds Validation	Automatically reject weight inputs below 0.1 kg or exceeding 1,500 kg per single delivery transaction to eliminate human typos.
FR 3.0	Automated Receipt Notification	The system shall execute instantaneous text receipt transmissions directly to smallholders upon collection updates.
FR 3.1	Text Notification Dispatch	Trigger an automated SMS receipt to the farmer’s mobile line the exact second a Collection Officer clicks "Submit".
FR 3.2	Audit Receipt Generation	Populate the SMS message containing the cooperative name, timestamp, exact crop weight, and running seasonal total.
FR 4.0	Inalterable Batch Aggregation	The database system shall compute storage totals dynamically to eliminate administrative manipulation.
FR 4.1	Mathematical Lock Calculation	Automatically compute total batch weights as a direct mathematical sum of individual farmer entries; direct edits are disabled.

FR 4.2	Discrepancy Flagging	Automatically cross-examine aggregate weights against external buyer invoices and flag anomalies directly to Super-Admins.
FR 5.0	Bulk Market Sale Logging	The system shall allow authorized management units to document high-volume merchant transactions securely.
FR 5.1	Wholesaler Contract Entry	Provide Cooperative Managers an interface to record batch wholesale deals, including total weight, buyer details, and price.
FR 5.2	Inventory Deduction Mapping	Automatically deduct sold quantities from the cooperative's active inventory metrics upon verification of bank transfer logs.
FR 6.0	Algorithmic Revenue Distribution	The platform shall compute individual payout models dynamically based on recorded performance percentages.
FR 6.1	Percentage Split Calculation	Compute individual farmer shares by dividing their documented weight by the total batch weight multiplied by net payout revenue.

FR 6.2	Ledger Audit Trail Export	Generate cryptographic financial distribution files mapping every RWF calculation step for regulatory submission to the RCA.
FR 7.0	Agronomic & Veterinary Mapping	The platform shall compile localized health indicators to facilitate fast preventative health interventions.
FR 7.1	Anomaly Flagging Input	Allow Collection Officers and Vets to record crop health markers or disease indicators during collection tasks.
FR 7.2	Regional Anomaly Heatmapping	Render an analytical GIS heatmap inside the Veterinarian portal plotting active disease clusters across geographic sectors.

5. Other Nonfunctional Requirements

5.1 Non-Functional Requirements Table

The non-functional boundaries of the platform ensure complete security, high availability, and absolute performance equity across the system footprint:

Requirement Type	Req ID	Technical Specification and Description
Security	**NFR 1**	Role-Based Access Separation: The application shall strictly restrict route entry based on permissions. A Collection Officer

		can never view financial profit boards, and a Cooperative Manager cannot alter raw crop drop-off inputs.
Performance	**NFR 2**	Dashboard Loading Limits: Core analytical database queries within the React manager dashboard shall return processed visualizations in under 1.5 seconds under standard broadband conditions.
Availability	**NFR 3**	API Hub Uptime Target: The central Django REST application service layer shall achieve a 99.9% operational runtime availability footprint, excluding scheduled mid-night patch windows.
Auditability	**NFR 4**	Un-Alterable Action Ledgers: Every database write, profile lookup, and financial adjustment must create an immutable system log tracking the user identity, time signature, and source IP address.
Usability	**NFR 5**	Bilingual Localization Support: The complete USSD user matrix shall display entirely in Kinyarwanda, while administrative desktop portals support instant toggle options between English and Kinyarwanda.
Scalability	**NFR 6**	Concurrent Processing Capacity: The PostgreSQL data architecture shall successfully isolate and process up to 100

		concurrent incoming USSD string inputs simultaneously without introducing memory bottleneck stalls.
Data Integrity	**NFR 7**	Atomic Database Commit Rules: All transactional financial updates across the Django backend shall use explicit atomic wrappers. If a multi-step split-calculation errors out midway, the complete operation rolls back immediately to prevent balance fragmentation.

6. Appendix

Appendix A: Glossary

RCA (Rwanda Cooperative Agency): The regulatory government authority overseeing all registered cooperative organizations in Rwanda.

USSD (Unstructured Supplementary Service Data): A GSM telecommunications text protocol that allows character strings to be sent between basic cell phones and application gateway servers without needing data packets.

Single Source of Truth: A data structural standard that ensures all system information comes from a single, verified database, preventing discrepancies across different ledger models.

Cooperative Collection Point: A physical site where smallholder farmers bring their crops to be weighed, checked, and logged.

Side-Selling: A breakdown of cooperative loyalty in which farmers sell directly to private traders, bypassing official channels due to low trust or long payment delays.

Appendix B: Analysis Models

To guide implementation, the PostgreSQL relational schema is designed around clear data connections, ensuring that total aggregates can always be traced back to individual, un-deletable entries.

Core Structural Integrity Rules Embedded in Database Core:

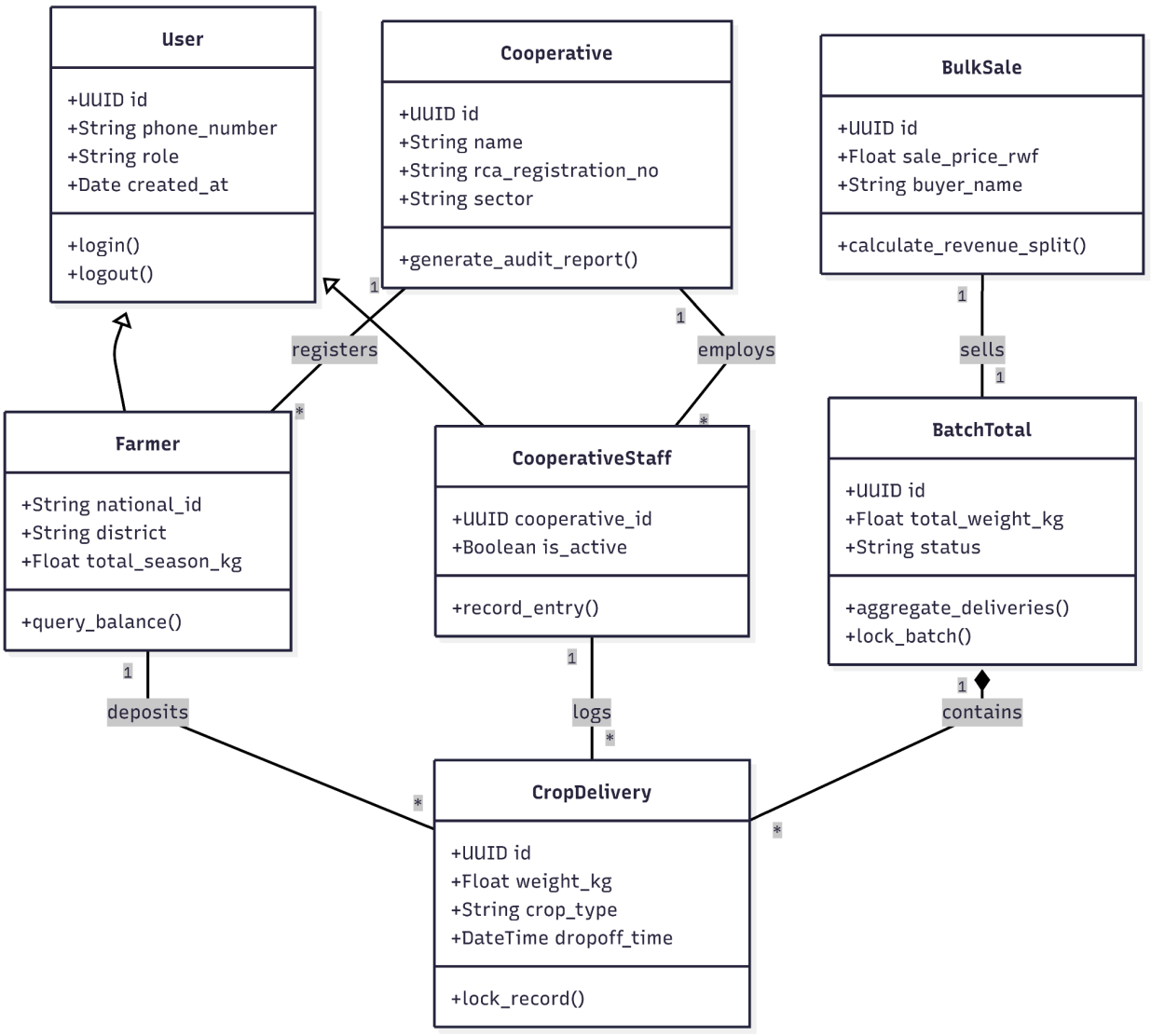
The Fraud Block: The CROP DELIVERY table uses an append-only transaction framework. Django controllers will block any SQL DELETE or UPDATE operations that target written transactions.

The Bottom-Up Link: The BATCH TOTAL record calculates its internal weight values by running a PostgreSQL SUM query across all CROP DELIVERY items linked to its unique ID. This setup prevents any cooperative operator from entering a false bulk number.

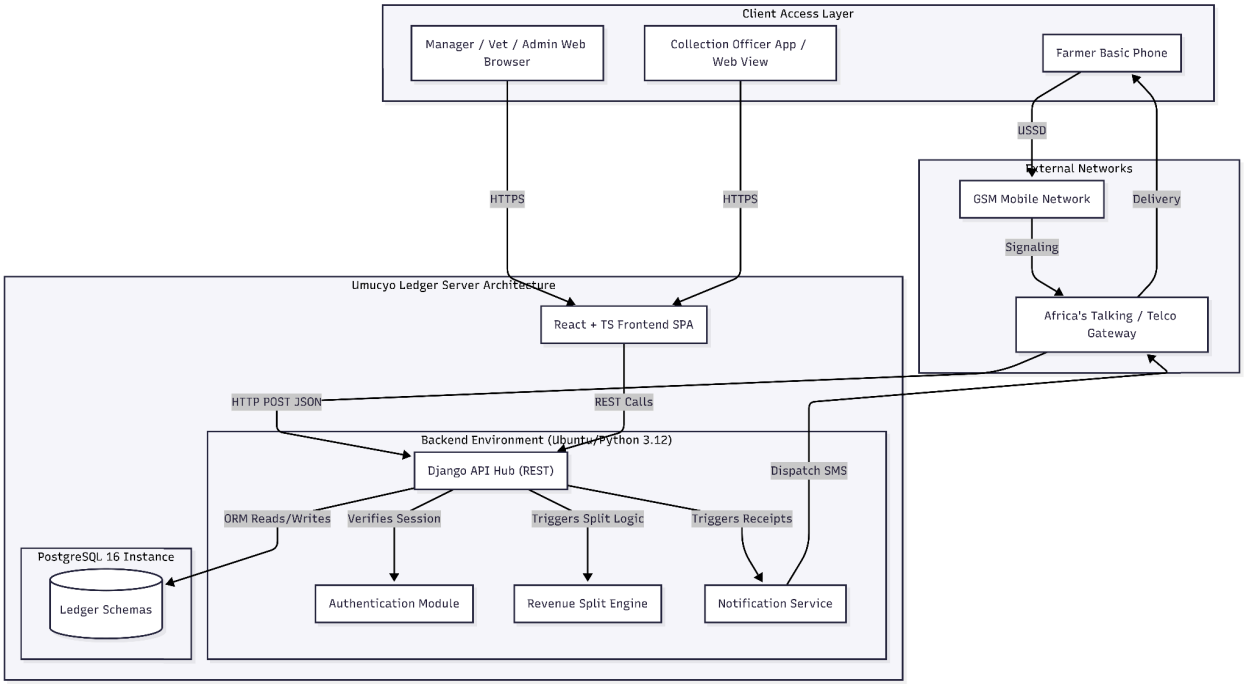
Software Architecture Design - UML diagrams

Structure diagrams

1. **Class diagram:** Since I use Postgres in this system, in this diagram, It demonstrates how tables and relationships are connected through a schematic diagram. Schema logic, like how CropDerively rolls up into BatchTotal to create database rules. It also shows when a table inherits somethingg from another.

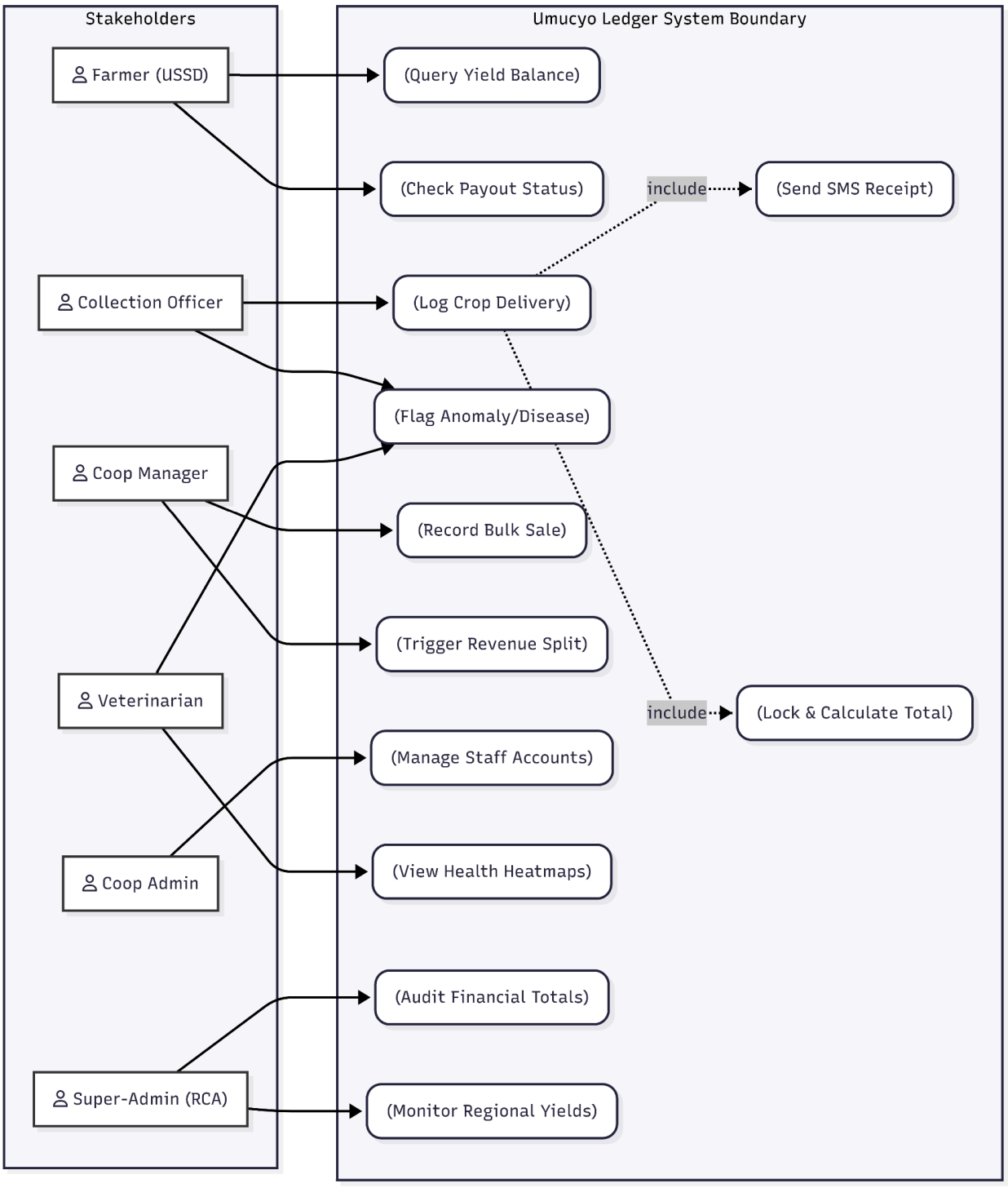

User
+UUID id
+String phone_number
+String role
+Date created_at
+login()
+logout()
Cooperative
+UUID id
+String name
+String rca_registration_no
+String sector
+generate_audit_report()
BulkSale
+UUID id
+Float sale_price_rwf
+String buyer_name
+calculate_revenue_split()
registers
employs
sells
Farmer
+String national_id
+String district
+Float total_season_kg
+query_balance()
CooperativeStaff
+UUID cooperative_id
+Boolean is_active
+record_entry()
BatchTotal
+UUID id
+Float total_weight_kg
+String status
+aggregate_deliveries()
+lock_batch()
deposits
logs
contains
CropDelivery
+UUID id
+Float weight_kg
+String crop_type
+DateTime dropoff_time
+lock_record()
1
*

2. **Component diagram:** This diagram shows components that our system need to be full function. It shows how the USSD network and Web interfaces all route through your centralized Django API, Postgres database, and other few components that can be used in this system.

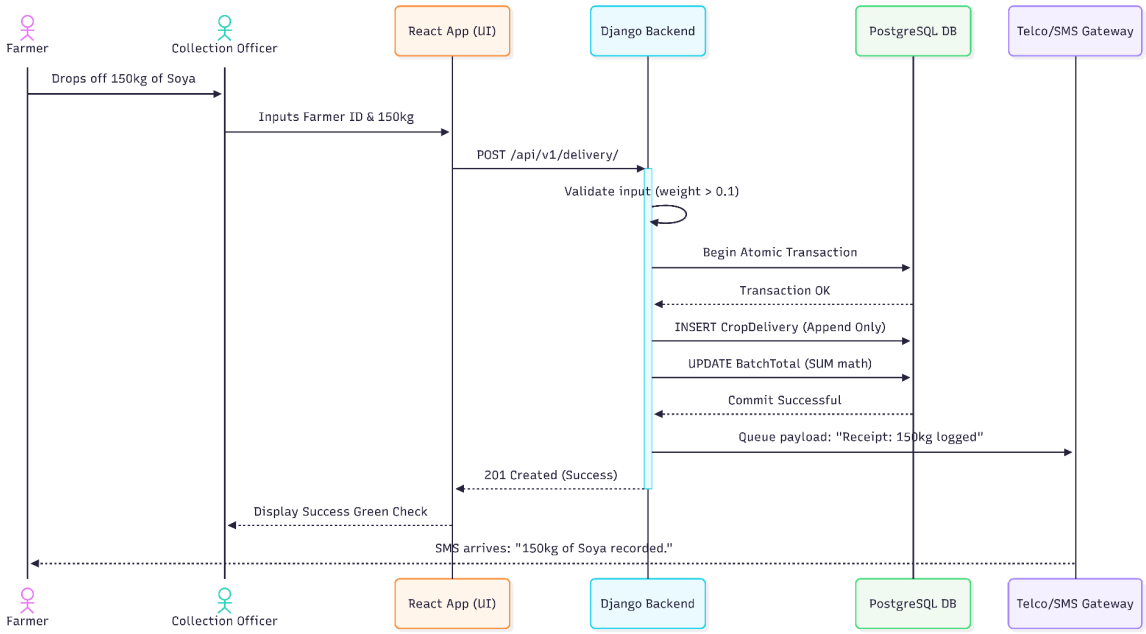


Behavioral diagram

3. **Use case diagram:** In this diagram, we map out exactly what each of the 6 user roles can do, highlighting how the core automated systems intercept human actions to prevent fraud. You can see how cooperative manager can record bulk sales and trigger revenue split



4. **Sequence diagram**: In this diagram, we visualize user interaction with the system. That include sequence of events when farmer drops off crops to the cooperative or collection officer receiving harvest.


Farmer
Collection Officer
React App (UI)
Django Backend
PostgreSQL DB
Telco/SMS Gateway
Drops off 150kg of Soya
Inputs Farmer ID & 150kg
POST /api/v1/delivery/
Validate input (weight > 0.1)
Begin Atomic Transaction
Transaction OK
INSERT CropDelivery (Append Only)
UPDATE BatchTotal (SUM math)
Commit Successful
Queue payload: "Receipt: 150kg logged"
201 Created (Success)
Display Success Green Check
SMS arrives: "150kg of Soya recorded."
Farmer
Collection Officer
React App (UI)
Django Backend
PostgreSQL DB
Telco/SMS Gateway